

Q1(A): Conference Entry Audit System

A technical conference records the badge ID of every person entering an exhibition hall. During a busy session, the same badge may appear several times because participants are allowed to leave and reenter.

The organizers also need to perform several badge-verification requests after the event. Develop a program that performs the following:

1. Read N badge IDs and preserve their original entry order.
2. Identify every badge ID that occurs more than once. For every such badge, display the badge ID and number of occurrences. Each repeated badge ID must appear only once in this report, and the report must follow the order in which the badge ID first appeared.
3. Create a second copy of the badge IDs arranged from smallest to largest.
4. Read Q badge IDs to be verified.
5. For every requested badge ID, use two different lookup implementations covered in the practical syllabus: one operating on the original sequence and one operating on the arranged sequence.
6. For each requested badge ID, display the required result and the number of comparisons made by both methods.
7. After all Q requests have been processed, automatically display the total comparisons by both methods and identify the better method for the given test data.
8. Correctly handle empty input, one badge, all IDs identical, no repeated IDs, requested ID absent, and first/last element requests.

Code:

```
import java.util.*;
```

```
public class q1 {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        if (!sc.hasNextInt()) {
```

```
            System.out.println("No badge records provided.");
```

```
    return;  
}
```

```
int n = sc.nextInt();  
if (n < 0) {  
    System.out.println("Number of badge records cannot be negative.");  
    return;  
}
```

```
int[] original = new int[n];  
for (int i = 0; i < n; i++) {  
    if (!sc.hasNextInt()) {  
        System.out.println("Incomplete badge record input.");  
        return;  
    }  
    original[i] = sc.nextInt();  
}
```

```
System.out.print("Repeated badge IDs:");  
boolean hasRepeatedBadge = false;  
for (int i = 0; i < n; i++) {  
    boolean appearedBefore = false;  
    for (int j = 0; j < i; j++) {  
        if (original[i] == original[j]) {  
            appearedBefore = true;  
            break;  
        }  
    }  
}
```

```
if (!appearedBefore) {
    int count = 0;
    for (int j = 0; j < n; j++) {
        if (original[i] == original[j]) {
            count++;
        }
    }
    if (count > 1) {
        System.out.print(" " + original[i] + " - " + count + " occurrences");
        hasRepeatedBadge = true;
    }
}

if (!hasRepeatedBadge) {
    System.out.print(" None");
}

int[] arranged = Arrays.copyOf(original, n);
Arrays.sort(arranged);
System.out.println("\nArranged badge IDs: " + Arrays.toString(arranged));

if (!sc.hasNextInt()) {
    System.out.println("No verification requests provided.");
    return;
}

int q = sc.nextInt();
if (q < 0) {
```

```
System.out.println("Number of verification requests cannot be negative.");
return;
}

long totalMethod1 = 0;
long totalMethod2 = 0;

for (int i = 0; i < q; i++) {
    if (!sc.hasNextInt()) {
        System.out.println("Incomplete verification request input.");
        return;
    }

    int badgeld = sc.nextInt();

    SearchResult linearResult = linearSearch(original, badgeld);
    SearchResult binaryResult = binarySearch(arranged, badgeld);
    totalMethod1 += linearResult.comparisons;
    totalMethod2 += binaryResult.comparisons;

    System.out.println("\nBadge ID: " + badgeld);
    System.out.println(linearResult.found ? "Found" : "Not Found");

    System.out.println("First position in original record: " + (linearResult.found ?
linearResult.index + 1 : -1));

    System.out.println("Number of comparisons -" + linearResult.comparisons);
    System.out.println("Number of comparisons -" + binaryResult.comparisons);
}
```

```
System.out.println("\nTotal comparisons - Method 1: " + totalMethod1);
```

```
System.out.println("Total comparisons - Method 2: " + totalMethod2);
```

```
if (totalMethod1 < totalMethod2) {
```

```
    System.out.println("Better method for this test data:");
```

```
}
```

```
else if (totalMethod2 < totalMethod1) {
```

```
    System.out.println("Better method for this test data:");
```

```
}
```

```
else {
```

```
    System.out.println("Better method for this test data: Both methods are equal");
```

```
}
```

```
}
```

```
private static SearchResult linearSearch(int[] values, int target) {
```

```
    int comparisons = 0;
```

```
    for (int index = 0; index < values.length; index++) {
```

```
        comparisons++;
```

```
        if (values[index] == target) {
```

```
            return new SearchResult(true, index, comparisons);
```

```
        }
```

```
    }
```

```
    return new SearchResult(false, -1, comparisons);
```

```
}
```

```
private static SearchResult binarySearch(int[] values, int target) {
```

```
    int low = 0;
```

```
    int high = values.length - 1;
```

```
int comparisons = 0;

while (low <= high) {
    int middle = low + (high - low) / 2;
    comparisons++;
    if (values[middle] == target) {
        return new SearchResult(true, middle, comparisons);
    }
    if (values[middle] < target) {
        low = middle + 1;
    } else {
        high = middle - 1;
    }
}

return new SearchResult(false, -1, comparisons);
}

private static class SearchResult {
    private final boolean found;
    private final int index;
    private final int comparisons;

    private SearchResult(boolean found, int index, int comparisons) {
        this.found = found;
        this.index = index;
        this.comparisons = comparisons;
    }
}
}
```

OUTPUT:

```

PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\CIE> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\066655f423bbfe63653f50105bf42b33\redhat.java\jdt_ws\CIE_e0bbb7c4\bin' 'q1'
5
101 103 105 106 107
Repeated badge IDs: None
Arranged badge IDs: [101, 103, 105, 106, 107]
102
103

Badge ID: 103
Found
First position in original record: 2
Number of comparisons -2
Number of comparisons -3
102

Badge ID: 102
Not Found
First position in original record: -1
Number of comparisons -5
Number of comparisons -3

```

Q2(B) Technical Support Ticket Navigator

A software company maintains active technical-support tickets dynamically. Every ticket contains Ticket_ID and Priority, where priority is between 1 and 5. The support engineer works with one ticket at a time and must be able to move both forward and backward through the active tickets.

New tickets must always follow this business rule: higher-priority tickets appear before lower-priority tickets; if two tickets have the same priority, the ticket received earlier remains first.

Develop a dynamically allocated program supporting these commands:

NEW id priority – Create a new ticket and place it in its correct position.

NEXT – Move the engineer to the next ticket, if one exists.

PREVIOUS – Move the engineer to the previous ticket, if one exists.

RESOLVE id – Remove the specified ticket from the active ticket collection.

CHANGE id newPriority – Change the priority of the specified ticket and reposition it so that all business rules remain satisfied.

SHOW – Display all active tickets from first to last as Ticket_ID and Priority.

SHOW_REVERSE – Display all active tickets from last to first.

CURRENT – Display the ticket currently selected by the engineer.

Additional Programming Requirement

Maintain a variable that counts the number of link modifications performed during every NEW, RESOLVE, and CHANGE operation. After each such operation, display the number of link modifications. At the end of execution, display the total number of link modifications.

The program must correctly handle:

- initially empty collection and only one ticket
- highest- and lowest-priority insertion
- several tickets with the same priority
- resolving the first, last, or current ticket
- changing a ticket from low to high priority and high to low priority
- invalid Ticket_ID
- movement beyond the first or last ticket

CODE:

```
import java.util.*;
```

```
public class q2 {
```

```
    static class Ticket {
```

```
        int id, priority;
```

```
        Ticket next, previous;
```

```
        Ticket(int id, int priority) {
```

```
            this.id = id;
```

```
            this.priority = priority;
```

```
        }
```

```
    }
```

```
    static Ticket head, tail, current;
```

```
    static int totalLinks = 0;
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    while (sc.hasNextLine()) {  
        String line = sc.nextLine().trim();  
        if (line.length() == 0) {  
            continue;  
        }  
  
        String[] part = line.split(" ");  
        String command = part[0].toUpperCase();  
  
        if (command.equals("NEW") && part.length == 3) {  
            int links = insert(new Ticket(Integer.parseInt(part[1]), Integer.parseInt(part[2])));  
            showLinks("NEW", links);  
        } else if (command.equals("NEXT")) {  
            if (current != null && current.next != null) current = current.next;  
            showCurrent();  
        } else if (command.equals("PREVIOUS")) {  
            if (current != null && current.previous != null) current = current.previous;  
            showCurrent();  
        } else if (command.equals("RESOLVE") && part.length == 2) {  
            Ticket ticket = find(Integer.parseInt(part[1]));  
            if (ticket == null) {  
                System.out.println("Invalid Ticket_ID");  
            } else {  
                int links = remove(ticket);  
            }  
        }  
    }  
}
```

```
        showLinks("RESOLVE", links);
    }
} else if (command.equals("CHANGE") && part.length == 3) {
    Ticket ticket = find(Integer.parseInt(part[1]));
    if (ticket == null) {
        System.out.println("Invalid Ticket_ID");
    } else {
        boolean wasCurrent = ticket == current;
        int links = remove(ticket);
        ticket.priority = Integer.parseInt(part[2]);
        links += insert(ticket);
        if (wasCurrent) current = ticket;
        showLinks("CHANGE", links);
    }
} else if (command.equals("SHOW")) {
    show(false);
} else if (command.equals("SHOW_REVERSE")) {
    show(true);
} else if (command.equals("CURRENT")) {
    showCurrent();
} else if (command.equals("END")) {
    break;
}

System.out.println("Total link modifications: " + totalLinks);
}
```

```
static int insert(Ticket ticket) {  
    int links = 0;  
    ticket.next = null;  
    ticket.previous = null;  
  
    if (head == null) {  
        head = tail = current = ticket;  
        totalLinks += links;  
        return links;  
    }  
  
    Ticket after = head;  
    while (after != null && after.priority >= ticket.priority) {  
        after = after.next;  
    }  
  
    if (after == head) {  
        ticket.next = head;    links++;  
        head.previous = ticket; links++;  
        head = ticket;        links++;  
    } else if (after == null) {  
        ticket.previous = tail; links++;  
        tail.next = ticket;    links++;  
        tail = ticket;        links++;  
    } else {  
        ticket.previous = after.previous; links++;  
        ticket.next = after;        links++;  
        after.previous.next = ticket; links++;  
    }  
}
```

```
        after.previous = ticket;    links++;
    }
    totalLinks += links;
    return links;
}
```

```
static int remove(Ticket ticket) {
    int links = 0;
    if (ticket == current) {
        current = ticket.next != null ? ticket.next : ticket.previous;
    }
    if (ticket.previous == null) {
        head = ticket.next;    links++;
    } else {
        ticket.previous.next = ticket.next; links++;
    }
    if (ticket.next == null) {
        tail = ticket.previous;    links++;
    } else {
        ticket.next.previous = ticket.previous; links++;
    }
    ticket.next = ticket.previous = null;
    totalLinks += links;
    return links;
}
```

```
static Ticket find(int id) {
    Ticket p = head;
```

```
while (p != null) {  
    if (p.id == id) return p;  
    p = p.next;  
}  
return null;  
}
```

```
static void show(boolean reverse) {  
    Ticket p = reverse ? tail : head;  
    while (p != null) {  
        System.out.println(p.id + " " + p.priority);  
        p = reverse ? p.previous : p.next;  
    }  
}
```

```
static void showCurrent() {  
    if (current == null) System.out.println("No current ticket");  
    else System.out.println("Current: " + current.id + " " + current.priority);  
}
```

```
static void showLinks(String operation, int links) {  
    System.out.println(operation + " link modifications: " + links);  
}  
}
```

OUTPUT:

```
sages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\066655f423bbfe63653f50105bf42b33\redhat.java
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\CIE> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-pr
sages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\066655f423bbfe63653f50105bf42b33\redhat.java
● NEW 101 2
NEW 102 5
NEW 103 2
SHOW
CURRENT
NEXT
CURRENT
RESOLVE 102
SHOW
NEW link modifications: 0
NEW link modifications: 3
NEW link modifications: 3
102 5
101 2
103 2
Current: 101 2
Current: 103 2
Current: 103 2
RESOLVE link modifications: 2
101 2
103 2
END
Total link modifications: 8
○ PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\CIE>
```